

Git Workshop: Questions

23 hands-on questions plus a Git setup guide. Name: _____ Team: _____

How to work

- Do everything inside a folder called **git-practice** (create it on your Desktop). It holds every project in these questions.
- Create and edit files with a **text editor** (VS Code or Notepad), not with shell tricks like `echo >`. That keeps Windows and Mac behaving the same.
- Do the question, then show your instructor the line marked **Show**. Tick the box and get an initial.
- Stuck? Run `git status`. It is always safe and it tells you what state you are in.

Part 0: Set up Git on your computer (do this once)

All you need today is **Git** and a **text editor** (VS Code or Notepad). You do **not** need a GitHub account.

0.1 Install Git

Windows: download **Git for Windows** from git-scm.com and accept the defaults (it includes Git Bash and Git Credential Manager).

Mac: run `xcode-select --install` in Terminal (or `brew install git`). Linux (Ubuntu/Debian): `sudo apt install git`.

Check it worked:

```
git --version
```

0.2 Tell Git who you are (required, once per computer)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Every commit you make is stamped with this name and email. Without it your first commit fails with *"Please tell me who you are"*. If you will use GitHub, use the same email as your GitHub account.

0.3 Make the default branch name main

```
git config --global init.defaultBranch main
```

0.4 Choose your editor (optional, avoids the scary vim window)

```
git config --global core.editor "code --wait"
```

That is for VS Code. If you ever get stuck in a strange text screen full of ~ characters, press Esc, type `:wq`, press Enter.

0.5 Line endings (optional)

```
git config --global core.autocrlf true      # Windows
git config --global core.autocrlf input    # Mac / Linux
```

0.6 Check your settings

```
git config --global --list
git config user.name
```

--global means "for every project on this computer". Run the same command inside one project without --global to change it for that project only.

Show: Show your instructor the output of `git config --global --list`: it must list your name, your email and `init.defaultbranch=main`.

Part 0B: Logging in to GitHub (optional today)

Git has no password of its own. You only log in when you talk to a **remote** such as GitHub. Today's questions use a folder on your own computer as the remote, so **skip this part if you have no GitHub account**.

GitHub no longer accepts your account password for Git commands. Use one of these instead:

- **Option 1, easiest: GitHub CLI.** Install it from cli.github.com, then run `gh auth login`. Choose GitHub.com, choose HTTPS, and sign in through the browser. Finish with `gh auth setup-git`.
- **Option 2: Git Credential Manager.** Included with Git for Windows. The first time you `git push` to a github.com address, a browser window opens so you can sign in, and Git remembers you. On Mac: `brew install --cask git-credential-manager`.
- **Option 3: personal access token.** On GitHub go to Settings, Developer settings, Personal access tokens. Create one with the *repo* permission. When Git asks for a password, paste the **token** (not your account password).
- **Option 4: SSH key.** Run `ssh-keygen -t ed25519 -C "you@example.com"`, add the .pub file under GitHub Settings, SSH keys, test with `ssh -T git@github.com`, and use SSH addresses like `git@github.com:user/repo.git`.

Make Git remember you so it does not ask every time:

```
git config --global credential.helper manager      # Windows (usually already set)
git config --global credential.helper osxkeychain  # Mac
git config --global credential.helper 'cache --timeout=3600' # Linux, 1 hour
```

Check who you are logged in as: `gh auth status`.

Never type your password or token into `git config`, save it in a file inside a project, commit it, or share it. A token is a password. If one leaks, delete it on GitHub straight away.

Part A: Your first repository

Question 1: Create a repository ☐ done initials: _____

Inside `git-practice`, make a new folder called **my-project**, go into it, and turn it into a Git repository. Then find out which branch you are on.

Hint: `mkdir`, `cd`, `git init`, `git branch --show-current`

Show: `git status` runs without a "not a git repository" error, and the branch is called **main**.

Question 2: Ask Git what it sees ☐ done initials: _____

Create **notes.txt** with three lines of text. Run `git status`.

In your own words: what does Git call this file, and why?

Hint: `git status`

Show: The status output, plus your answer.

Question 3: Stage a file ☐ done initials: _____

Tell Git you want `notes.txt` in the next snapshot. Run `git status` again. What changed in the output?

Hint: `git add`

Show: `notes.txt` is listed under "Changes to be committed".

Question 4: Make your first commit ☐ done initials: _____

Save a snapshot with the message **Add notes**. Look at the history.

Hint: `git commit -m`, `git log --oneline`

Show: `git log --oneline` shows exactly one commit.

Question 5: Two files, two commits ☐ done initials: _____

Create **todo.txt** with three tasks. Stage it and commit it with the message **Add todo list**.

Explain in one sentence: what is the difference between *add* and *commit*?

Hint: `git add`, `git commit`

Show: `git log --oneline` shows two commits, and you explain *add* vs *commit*.

Part B: Looking at changes

Question 6: See what you changed ☐ done initials: _____

Edit `notes.txt`: change one existing line and add one new line. **Do not stage yet**. Run `git diff`.

Which lines start with `-` and which with `+`, and what does each mean? Then stage and commit the change with the message **Update notes**.

Hint: `git diff`, `git add`, `git commit`

Show: The diff output on screen, your explanation, and a third commit in `git log --oneline`.

Question 7: Undo a mistake before committing ☐ done initials: _____

Edit `todo.txt` and add a nonsense line. Change your mind: throw the edit away so the file goes back to its last saved version.

Hint: `git restore`

Show: `git status` says nothing to commit, and `todo.txt` is back to normal.

Question 8: Inspect a commit ☐ done initials: _____

Look at the history, then use one command to see the full details of your **Add notes** commit: who wrote it, when, and which lines it added.

Hint: `git log --oneline, git show`

Show: The `git show` output on screen. The author name is the one you set in Part 0.

Part C: Branches and merging

Question 9: Create and use a branch ☐ done initials: _____

Create a branch called **feature-menu** and switch onto it. Create **menu.txt** with three menu items and commit it with the message **Add menu**.

Hint: `git switch -c` does both steps at once

Show: `git branch` shows a `*` next to `feature-menu`, and `git log --oneline` shows the new commit.

Question 10: Branches keep work separate ☐ done initials: _____

Switch back to **main** and look in the folder: is `menu.txt` there? Switch to `feature-menu` again: is it back? Explain what is happening.

Hint: `git switch`

Show: You demonstrate the file disappearing and reappearing, and explain why.

Question 11: Merge a branch ☐ done initials: _____

Switch to `main` and merge `feature-menu` into it. Did Git say *Fast-forward*? Explain why no new merge commit was needed. Then delete the finished branch.

Hint: `git merge, git branch -d`

Show: The merge output and your explanation. `menu.txt` is now on `main`.

Question 12: Cause and fix a merge conflict ☐ done initials: _____

Follow these steps in order:

1. On `main`, create **greeting.txt** with one line: `Hello, world`. Commit it with the message **Add greeting**.
2. Create a branch **formal**. Change the line to `Good day, world` and commit.
3. Switch back to `main`. Create a branch **casual**. Change the **same line** to `Hey, world` and commit.
4. While on `casual`, merge `formal` into it. Git will report a **CONFLICT**.
5. Open `greeting.txt`. Find the lines with `<<<<<<, =====` and `>>>>>>`. Keep the text you want and delete those three marker lines.
6. Stage the file and finish the merge with a commit (use `-m` so no editor opens).
7. Switch back to `main`.

Hint: `git merge, git add, git commit -m`

Show: `git status` is clean, `greeting.txt` contains no marker lines, and `git log --oneline --graph` shows the merge.

Part D: Undo and pause

Question 13: Undo a commit safely ☐ done initials: _____

On main, create **price.txt** containing Coffee: \$40 (a typo!) and commit it with the message **Add price**.

Undo that commit **without erasing history**: Git should add a new commit that cancels it out.

Hint: git revert (add --no-edit to skip the editor)

Show: git log --oneline shows both the original commit and a Revert commit, and price.txt is gone.

Question 14: Pause your work ☐ done initials: _____

Edit notes.txt but do not commit. Your teacher says "drop everything": put your unfinished work aside so your folder is clean, then bring it back.

Hint: git stash, git stash pop

Show: git status is clean after the stash, and your edit is back after the pop.

Part E: Working with a remote (no GitHub needed)

Question 15: Create a remote and push ☐ done initials: _____

From inside git-practice run git init --bare shared.git. A *bare* repository is a shared copy with no working files. It plays the part of GitHub.

In my-project, connect it under the name **origin** and push main to it, remembering the link.

Hint: git remote add origin ../shared.git, git push -u origin main

Show: git remote -v lists origin, and git branch -r lists origin/main.

Question 16: Clone as a teammate ☐ done initials: _____

From inside git-practice, clone shared.git into a new folder called **teammate**. Pretend it is a classmate's laptop. Check that it has all your files and the same history.

Hint: git clone

Show: git log --oneline inside teammate matches my-project.

Question 17: The teammate pushes a change ☐ done initials: _____

Inside teammate: edit notes.txt and add the line Added by teammate. Commit it and push it to the remote.

Hint: git commit -am, git push

Show: The push succeeds and git log --oneline inside teammate shows the new commit.

Question 18: Fetch without merging ☐ done initials: _____

Go back to my-project. Download the teammate's work **without** changing your files. Then compare your main with origin/main. What is different?

Hint: git fetch, git log main --oneline, git log origin/main --oneline

Show: origin/main has one commit that main does not, and your notes.txt is unchanged.

Question 19: Pull ☐ done initials: _____

Now bring the teammate's change into your main branch. Explain how *pull* differs from *fetch*.

Hint: git pull

Show: notes.txt contains the teammate's line and git log --oneline shows their commit.

Question 20: Push a new branch ☐ done initials: _____

Create a branch **feature-hours**, add **hours.txt**, commit it, and push the branch to the remote, setting the link at the same time.

Hint: `git push -u origin feature-hours`

Show: `git branch -r` now lists `origin/feature-hours`.

Part F: Extras

Question 21: Tag a release ☐ done initials: _____

Switch back to main. Create a **lightweight** tag called **v1.0** and an **annotated** tag called **v1.1** with the message **Second release**.

Hint: `git tag`, `git tag -a`

Show: `git tag` lists both, and `git show v1.1` displays the message.

Question 22: Stop tracking a file by mistake ☐ done initials: _____

Create **debug.log** with any text and commit it (pretend it was an accident). Now make Git **stop tracking** it **without deleting the file**, and make sure Git ignores it from now on.

Hint: `.gitignore`, `git rm --cached`

Show: `debug.log` is still in your folder. After you edit it, `git status` does not mention it.

Question 23: Cherry-pick one commit ☐ done initials: _____

Create a branch **experiment** and make **two** commits on it: one adding **a.txt**, one adding **b.txt**. Switch to main and bring over **only** the `b.txt` commit.

Hint: `git log --oneline` to find the hash, then `git cherry-pick`

Show: `b.txt` exists on main but `a.txt` does not.

Quick check: answer in your own words

1. Name the four places your work can be while using Git.

2. What is the difference between `git add` and `git commit`?

3. What is the difference between `git fetch` and `git pull`?

4. Why does `git revert` add a new commit instead of deleting the old one?

5. What is a merge conflict, and who fixes it?

6. Why do we set `user.name` and `user.email`?
